Bridge Inspector Field Guide

Setting up a georeferenced IFC on site, running the AR overlay, and using the inspection tools.

Application build **v211**

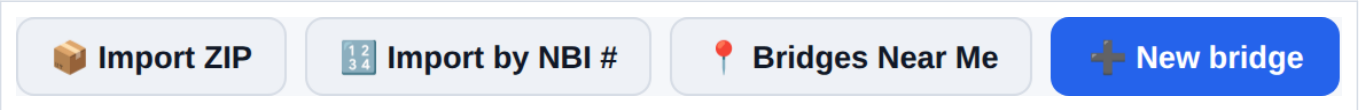
Installed as a PWA. Works offline once cached.

Contents

SECTION 1

Bridges: the unit of work

Nothing in the app is loose. Every photo, sketch, scan set, tag, IFC model, CAD overlay and report belongs to a **bridge**, and you work inside one at a time. The first screen is the list of them.



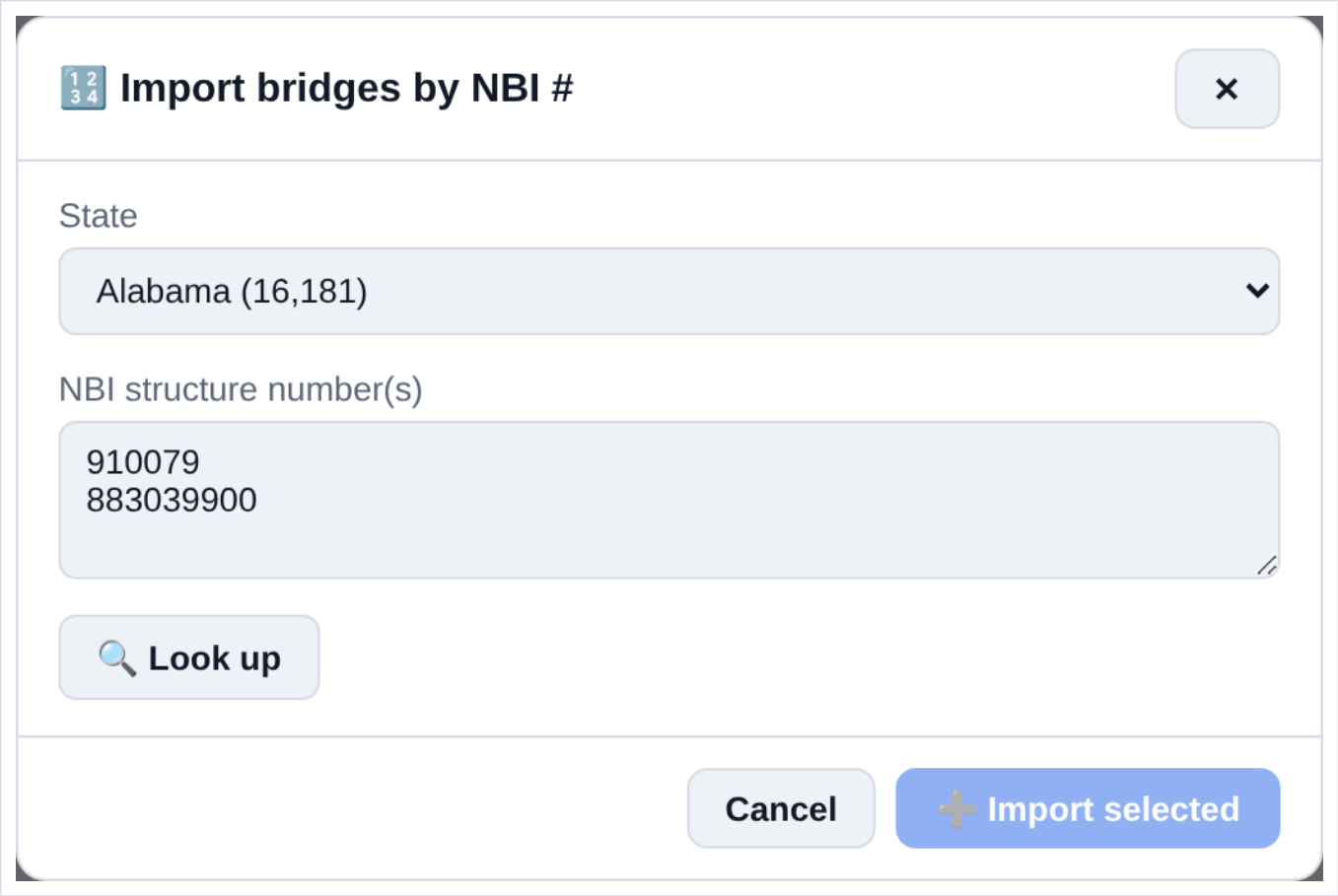
The four ways to get a bridge into the app.

Four ways in

- **New bridge** — a blank one with a title and description you type. Fine for a structure you are only visiting once.
- **Import by NBI #** — pull it straight out of the National Bridge Inventory. The fastest correct option when you have the structure number.
- **Bridges Near Me** — for when you do not. Uses your GPS and the NBI coordinates to list what is around you.
- **Import ZIP** — restore a bridge someone else archived, or one from a previous device. This is the lossless path: photos, tags, comments, locations, headings, AprilTag detections, annotations and the CAD overlay all come back.

Importing from the NBI

1. Tap **Import by NBI #**.
2. Pick the state. The count beside each is how many structures it holds.
3. Paste the structure number, or several — one per line or comma separated.
4. Tap **Look up**, check the matches, then **Import selected**.

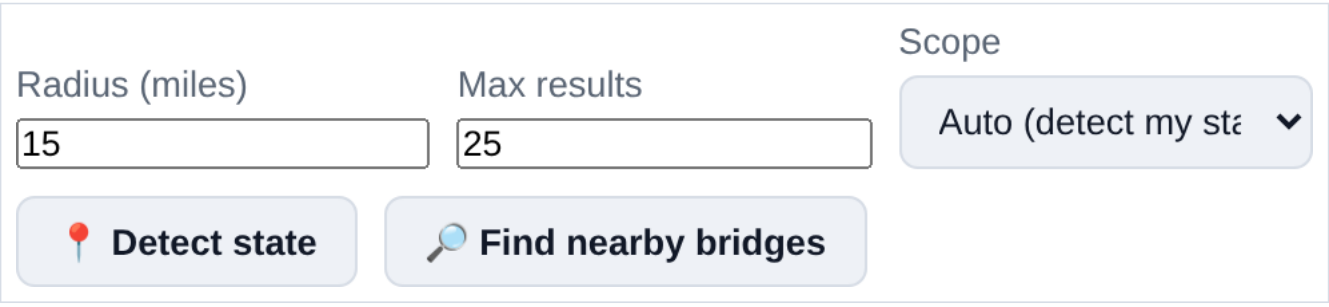


Several numbers at once is the normal case — build the day's route in one go before you leave.

An imported bridge arrives titled from the NBI record, described as *feature crossed · year built · NBI number*, and carries the inventory coordinates as its location — which is what puts it on the map before you have taken a single photo.

Finding what is around you

Bridges Near Me takes your current fix and searches the inventory around it. Set a radius in miles and a result cap; the scope defaults to auto-detecting your state, and widens to all states if you are near a line. Results come back on a map and as a list — pick one and import it.



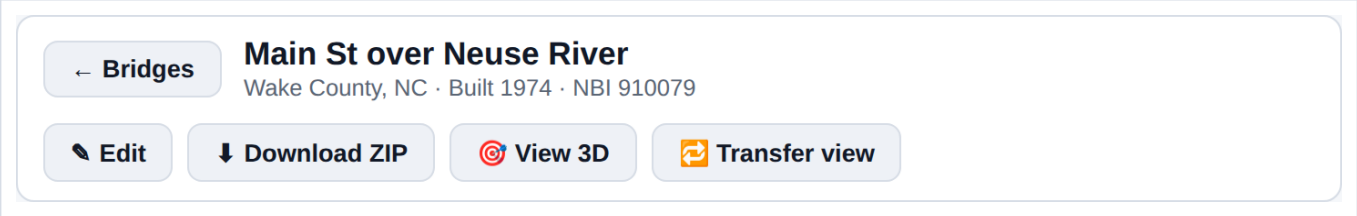
Radius, result cap and scope.

The inventory is bundled, not fetched

NBI data ships with the app as per-state files, so both of these work with no connection once the app is cached. The consequence is that it is a snapshot: as current as the build you are running, not as current as the FHWA's servers.

Working inside a bridge

Opening one swaps to the workspace. The banner across the top is where the bridge-level actions live.



Back to the list, edit the title and description, archive the whole thing to ZIP, open the 3D model, or open the device-to-device transfer panel.

Deleting a bridge deletes its contents

Removing a bridge removes its photos, sketches, scans and its stored IFC with it. The app confirms with the photo count in the prompt, but there is no undo — take a ZIP first if there is any doubt.

SECTION 2

Clearing the app cache

The app is a PWA. A service worker precaches every file it needs — HTML, CSS, JavaScript, the three.js and web-ifc libraries — so it keeps working with no signal under a bridge. The cost of that is that a browser reload does not necessarily fetch new code: the service worker answers from its cache first, and only swaps in a new cache when it sees a new build.

Clearing the cache forces a clean download of the whole app.

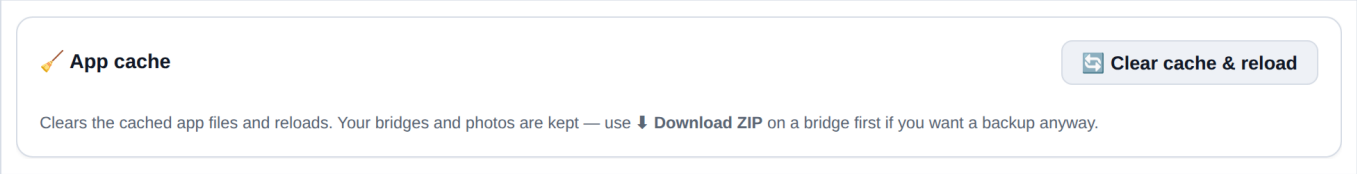
When to clear it

- A new build has been pushed and the version in the header has not changed. The build number is shown next to the app title in the header, as **build v211** plus a timestamp — check it against what you expect.
- A control described in this guide is missing, or a button does nothing.
- The app behaves inconsistently after an update — half-old, half-new code is the usual cause.
- Rendering or geometry looks wrong in a way that a reload does not fix.

It is not a fix for a bad GPS fix, a wrong heading, or a model in the wrong place. Those are covered in sections 4 to 7.

How to clear it

1. Open **Settings** from the header.
2. Find the **App cache** card at the top of the settings panel.
3. Tap **Clear cache & reload**.
4. Wait for the app to reload, then confirm the build number in the header has changed.



The App cache card in Settings. Clearing the cache does not touch your data.

Your data is safe

Bridges, photos, tags, annotations and scan sets live in IndexedDB, not in the file cache, so clearing the cache leaves all of them alone. If you want a backup anyway, use **Download ZIP** on a bridge before you clear.

Do this before you leave signal

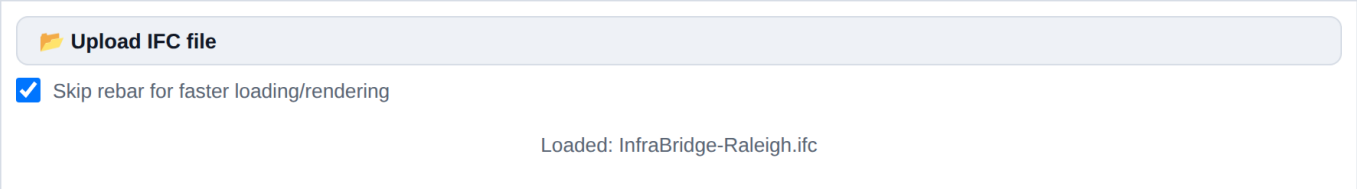
Clearing the cache requires a connection — it deletes the cached app and immediately re-downloads it. Do it in the truck or at the office, never standing under the structure you came to inspect.

SECTION 3

Attaching the IFC model

The IFC is loaded into the 3D viewer, and everything else — the AR overlay, element tagging, the plan view — reads from that one loaded model. Load it once per session.

1. Open the **3D View**.
2. Tap **Upload IFC file** and pick the .ifc from your device, or drag the file onto the viewer area.
3. Leave **Skip rebar for faster loading/rendering** ticked unless you specifically need reinforcement geometry. Rebar can be most of the element count in a bridge model and it is the difference between a model that loads on a phone and one that does not.
4. Wait for the status line to read **Loaded: <filename>**.



The upload controls. The status line underneath is the confirmation that the model is in — until it says Loaded, nothing downstream will work.

Georeferencing

For the AR overlay to put the model in the right place on the ground, the file has to say where in the world it is. The app reads, in this order:

- **IfcProjectedCRS** — which coordinate system the model's numbers are in.
- **IfcMapConversion** — the eastings/northings, rotation and scale that put the model's local origin into that system.
- **IfcSite** RefLatitude / RefLongitude / RefElevation — the fallback position, and the source of the model's ground elevation.

The projected CRS has to be one the app knows — it is a fixed list in the code, not a lookup. Currently: EPSG:7062 (Nebraska LDP, ftUS); EPSG:2264 and EPSG:32119 (North Carolina, ftUS and metric); EPSG:2263 and EPSG:32118 (New York Long Island, ftUS and metric); and EPSG:3418 and EPSG:26976 (Iowa South, ftUS and metric). A model in any other CRS will load and display, but it will not georeference, and the AR view will have nothing to anchor to — adding a zone is a code change, not a setting.

> **Test models**
>
> The repository ships georeferenced test files under **test-models/** — a 12 ft cube and the InfraBridge model, each built for a specific site and named for it. Use one of those to prove the workflow before you trust a production model on site.

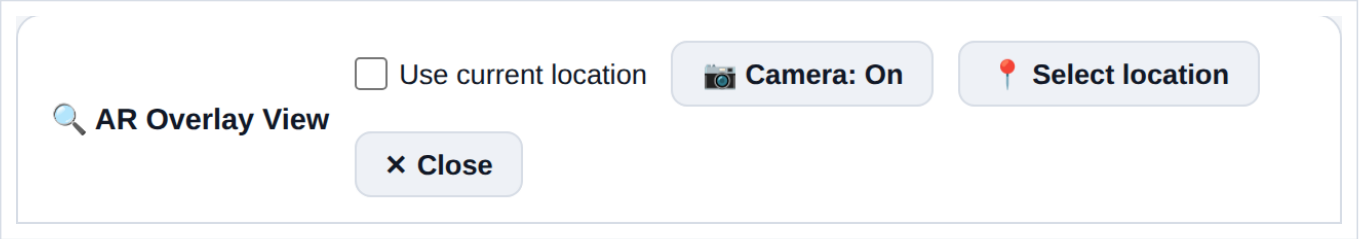
SECTION 4

Starting the AR view

The AR view draws the loaded model over the live camera feed, positioned from your GPS fix and oriented from the phone's compass and accelerometers. It is a sensor-driven overlay, not a tracked AR session, so it is only ever as good as the fix and the compass — which is what the controls in this section and the next two are for.

1. Load the IFC first (section 3). Without a model there is nothing to overlay.
2. From the main screen, tap **Augmented reality view**.
3. Allow camera access when prompted, and location access if you have not already.
4. Confirm the header reads **Camera: On**.

The entry point on the main screen.

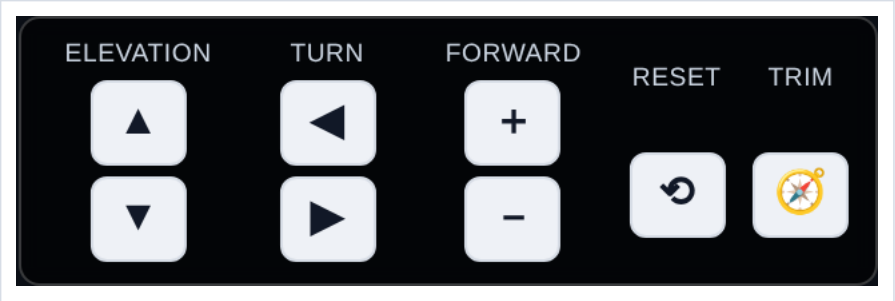


The AR header. **Use current location** drives the eye position from live GPS; **Select location** lets you drop a point on a map instead, which is how you rehearse a site from the office.

The on-screen controls

The view follows the phone by default. The control cluster at the bottom lets you nudge it when the sensors are not enough:

- **Elevation** — raise or lower the eye point.
- **Turn** — rotate the view left or right.
- **Forward** — step the eye point forward or back along the view direction.
- **Reset**, the circular arrow — discard every manual pan, elevation, turn and zoom so the view follows the phone again. It does not re-frame the model and it keeps your heading trim.
- **Trim**, the compass icon — clear the standing heading correction and go back to the raw compass.



Reset and Trim are deliberately separate. Reset undoes what you did this minute; Trim undoes the compass correction you calibrated earlier and probably want to keep.

Heading trim

Phone compasses read magnetic north and are pulled around by the steel you are standing next to. If the overlay is rotated off the real structure, turn it onto alignment with the **Turn** buttons — while the view is live, those nudges are folded into a saved heading trim rather than a temporary offset. The trim survives closing and reopening the view, so you calibrate once per site. Use **Trim** to throw it away when you move to a different structure.

One more thing has to be right before the overlay will hold still: the camera's field of view. That has a section of its own, next.

SECTION 5

Camera field of view

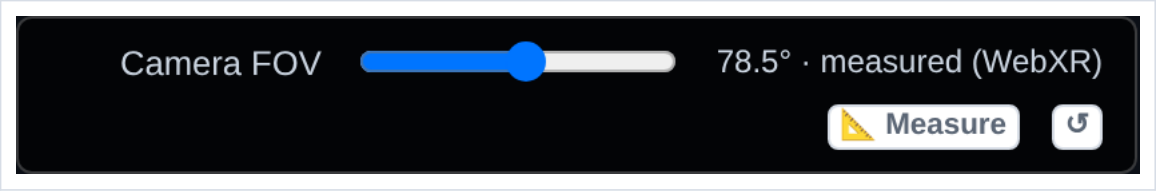
Position tells the app where you are. Heading tells it which way you are looking. Field of view tells it *how much of the world fits on the screen* — and it is the setting people skip, because a wrong FOV does not look wrong when you hold still. It only shows up when you move.

What a wrong FOV looks like

The renderer draws the model through a virtual camera. If that virtual camera's cone is wider than the real lens, everything in the overlay is drawn smaller and closer to the centre of frame than it should be; if it is narrower, larger and further out. Centre a pier and it will look fine. Then pan.

- **The overlay slides the way you pan and lags behind** — the virtual camera is too wide. The model appears painted on a pane of glass held in front of you rather than fixed to the world.
- **The overlay runs ahead of the world** — too narrow. Features at the edge of frame separate from their real counterparts first.
- **The centre lines up and the edges do not** — this is the tell.

That last point is the diagnostic worth remembering. A position error moves the whole model together. A heading error rotates it. An FOV error is zero at the centre of the screen and grows towards the edges — so if it sits dead on the structure in the middle of frame and splays apart at the corners, stop adjusting position and heading and fix the FOV.



The Camera FOV row in the heads-up panel. The read-out shows the angle in use and where it came from.

Measure it, do not guess

No web API reports a camera's field of view. `getUserMedia` hands back a video stream and nothing about the optics — `getSettings()` and `getCapabilities()` simply have no such member. That is why the app ships an 82° horizontal default, which is a spec-sheet figure for a typical main rear lens and not a measurement of yours.

Measure gets around it. WebXR does expose the real optics: an `immersive-ar` session hands back a projection matrix built from the device's own camera intrinsics, and the field of view falls straight out of two of its terms. The button opens a session for a moment, reads one frame, and closes it again. Do this once per phone — the value is saved.

- It needs a user gesture, which is why it is a button rather than something that happens automatically at start-up.
- On Android it needs ARCore (Google Play Services for AR). Without it the button reports *immersive-ar unsupported* and you fall back to the slider.
- The screen flashes into an AR session and straight back out. That is expected, not a crash.
- One caveat: this is the FOV of the *XR* view, which need not exactly equal the video stream's — the two can pick different sensor crops. It is a far better starting point than a spec sheet, but the slider is still there to trim.

Trimming it by hand

When Measure is unavailable, or afterwards to fine-tune, calibrate against the world:

1. Find a long straight edge you can identify in both the model and the real structure — a girder, a barrier, a deck joint.
2. Line the overlay up on it with the edge at the **centre** of the screen, using position and heading, not the FOV slider.
3. Pan slowly so that edge travels out towards the side of the screen.
4. If the overlay's edge falls behind the real one on the way out, the FOV is too wide — reduce it. If it runs ahead, increase it.
5. Repeat until the edge tracks from centre to corner without separating.

The circular-arrow button at the end of the row discards the saved value and returns to the 82° default.

The camera is pinned to 1.0×

The AR feed explicitly requests 1.0× zoom, which keeps it on the main lens. This matters more than it sounds: asking for the minimum available zoom would select the 0.5× ultrawide on a modern phone, whose field of view is nearer 120°. The 82° default would then be wildly wrong, and the overlay would swim about while every other setting was correct. If you trimmed the FOV slider at some earlier point, reset it and re-measure.

What the app derives from the number

What you set is the horizontal field of view of the lens. That is not what reaches the renderer — three corrections come first, which is why the read-out can differ from the figure you entered:

- **Zoom.** Zoom narrows what the lens shows. The feed is normally pinned at 1.0×, but a device may clamp or refuse that, and the render angle then has to follow.
- **Stream shape.** A spec FOV is quoted across the sensor's long axis — vertical for a portrait stream, horizontal for a landscape one — so the conversion uses the stream's native aspect ratio, not the screen's.
- **Cover crop.** The video is displayed with `object-fit: cover`, which crops whichever axis overflows. A display wider than the stream crops height, and cropped height changes the visible vertical field of view.

Only after all three does the result become the renderer's vertical FOV. The practical consequence: **a value trimmed in portrait is not automatically right in landscape**, because the crop differs. If you rotate the phone and the overlay stops tracking, that is why — trim it in the orientation you actually work in.

SECTION 6

Correcting for geoid height differences

Why the model floats or sinks

A phone's `getAltitude()` returns height above the **WGS84 ellipsoid** — a smooth mathematical figure. Survey and bridge plan elevations are **orthometric**, measured from the geoid, which is what NAVD88 approximates. The two differ by the geoid separation *N*, and across the continental US *N* is negative and large: roughly -22 m to -34 m.

The relationship is **H = h - N**, where *h* is the phone's ellipsoidal height and *H* is the orthometric elevation your model is drawn to. Uncorrected, the phone thinks it is roughly 30 m — about 100 ft — higher than it really is, and the model appears far below you.

The Ground datum row in the AR heads-up panel. It reads **uncalibrated** until you set an offset.

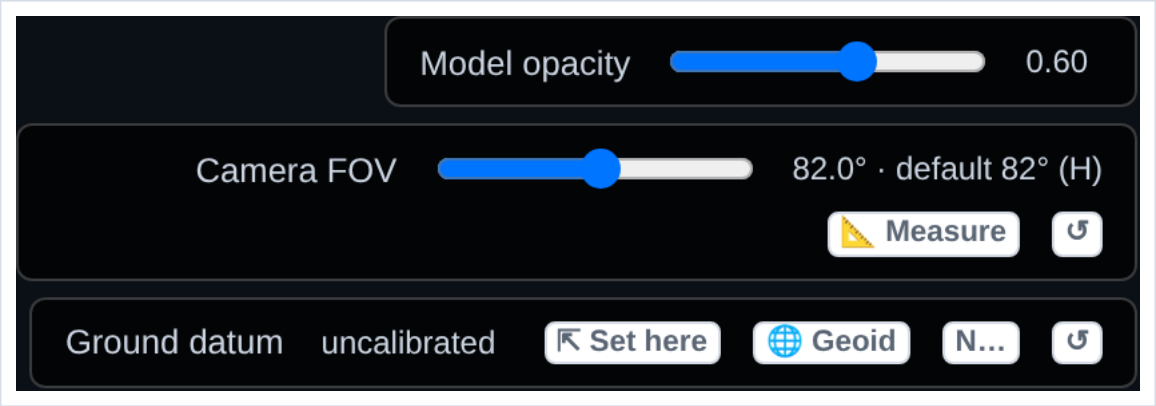
Three ways to fix it

All three write the same stored offset. It is regional — one value covers your whole working area — and it persists between sessions.

- **Set here** — the field method. Stand on ground whose true elevation you know, tap it, and enter that elevation in feet. The app shows what the phone currently reports and stores the difference. When the model's ground came from IfcSite, it pre-fills that as a suggestion.
- **Geoid** — looks up *N* for your position from the NOAA NGS geoid API. Needs a connection, and covers the US.
- **N…** — type the separation directly, in metres, negative across the continental US. Get it from any geoid calculator before you leave the office. This is the one that works with no signal.

The circular-arrow button at the end of the row clears the stored offset and goes back to the phone's raw altitude.

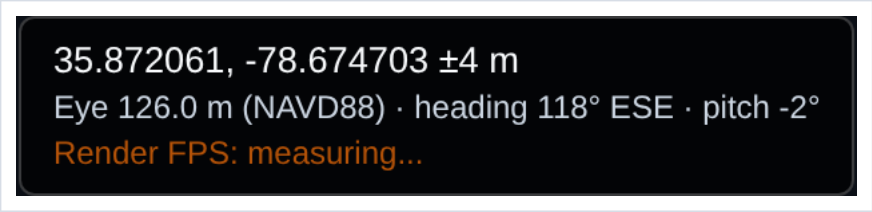
> **Use ground you can trust**
>
> "Set here" calibrates from the ground under your feet, so pick a benchmark, a known deck elevation, or a surveyed point — not a guess off a topo map. And note the app deliberately does *not* suggest the model's lowest geometry as your standing elevation: a model with piles or footings has its minimum well below grade, and accepting that would calibrate your datum underground.



The full heads-up panel: model opacity, camera FOV, and the ground datum row. It scrolls if it runs out of room.

Checking it worked

The readout at the top left shows your position and the eye elevation the app is actually using. After calibrating, that elevation should agree with the site's real elevation, and the model should sit on the ground rather than floating above or below it.



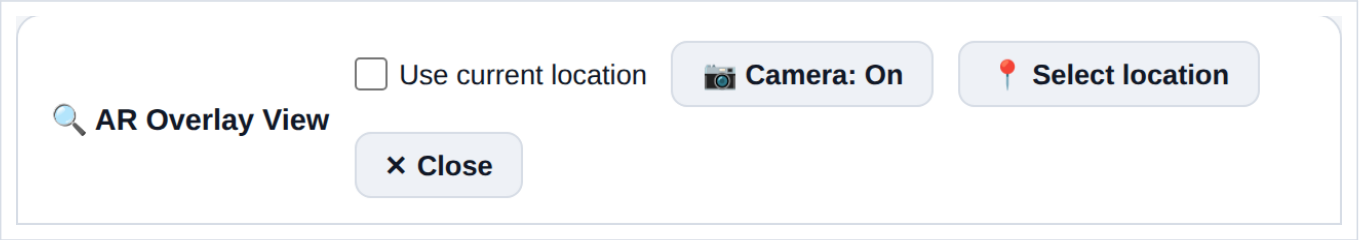
Position and eye elevation, top left of the AR view.

SECTION 7

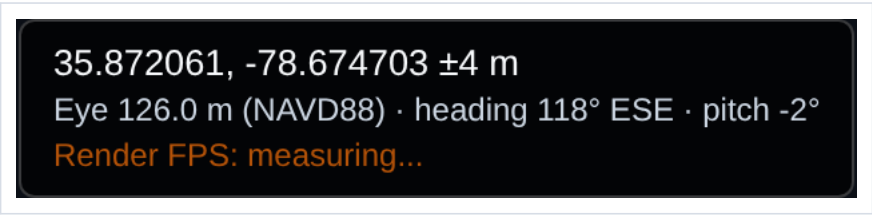
Turning on GPS

The AR view can take its eye position from live GPS or from a point you pick on a map. Live GPS is what you want on site.

1. Open the AR view.
2. Tick **Use current location** in the header.
3. Grant location permission if the browser asks. On Android, choose **Precise** — approximate location is useless for this.
4. Watch the readout at the top left. It will show your latitude, longitude and accuracy once a fix arrives.



Use current location is remembered — tick it once and it stays ticked the next time you open the AR view.



A good fix. The ± figure is the accuracy the phone reports; if it is tens of metres, the overlay will be tens of metres out.

If it will not pick up a fix

- The app requests a high-accuracy watch with a 30 second timeout and no cached positions, so the first fix outdoors can genuinely take half a minute. Give it time before deciding it has failed.
- Stand in the open, away from the structure, until the first fix lands. Under a deck or between piers you are shielded from most of the sky.

- Check the site permission in the browser — a denied location permission is silent from inside the page. In Chrome: the padlock in the address bar, then Permissions.
- Location requires a secure context. Over HTTPS or from localhost it works; over plain HTTP it will not.
- Turn the phone's own Location Services on. The browser cannot override it.

Working without GPS

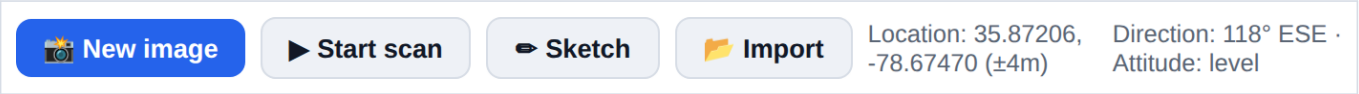
Untick **Use current location** and tap **Select location** to drop your eye position on a map instead. This is how you rehearse a site before you get there, and it is the fallback when the fix is too poor to use — a point you place by eye off a satellite image is often better than a 40 m fix.

The plan view

The plan view in the bottom left corner shows the model from above with your position and view direction on it. Drag the orange dot to move yourself; drag the green dot to change the direction you are looking. Moving your position leaves the view angle alone, and vice versa. Pinch to zoom. The **satellite** button toggles an Esri satellite basemap underneath, which makes it obvious whether the model is sitting where the structure actually is — it uses mobile data.

The capture screen readout

Outside the AR view, the main toolbar shows the same location and heading that will be written into the next photo you take.



Location, accuracy, direction and attitude, live on the main toolbar.

Part two

What the app can do

Everything in part one exists to get the model standing in the right place. This part covers what that buys you: photos that know which element they are of, condition ratings that go back into the IFC, photos that know where they were taken, measurements off a photograph with no tape, image sets a photogrammetry pipeline can use — and the report that comes out of all of it.

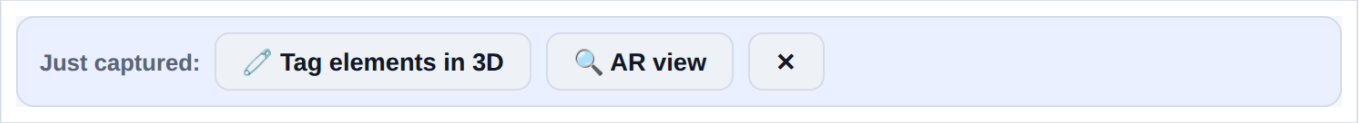
SECTION 8

Tagging photos to model elements

A photo of a crack is worth much more when the report knows it is *Pier 3 Column* and not just a photo of concrete. Tagging links a photo to one or more IFC elements, and the link is stored with the photo.

The fast path: straight after the shot

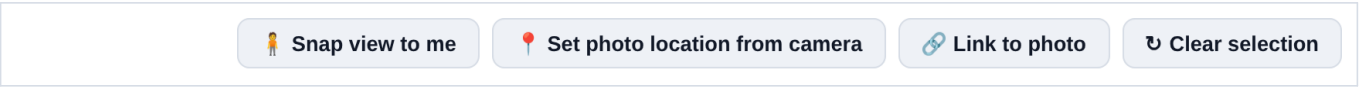
Take a picture and a follow-up bar appears under the toolbar. You do not have to go back through the gallery to find the photo you just took.



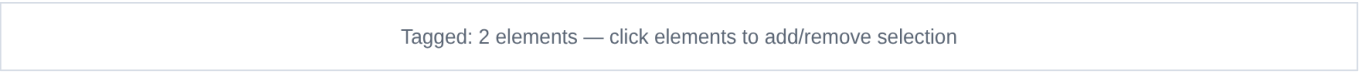
Tag elements in 3D opens the viewer with this photo already linked; **AR view** opens the overlay instead.

Selecting the elements

1. In the 3D viewer, tap **Snap view to me**. This puts the 3D camera where you are standing, looking the way the phone is pointing — so the model on screen matches what you were photographing, and you can recognise the element you want.
2. Click elements in the model to add them to the selection. Click again to remove one. The status line keeps a running count.
3. Tap **Tag selected elements to photo**.



The tagging row at the bottom of the 3D viewer.

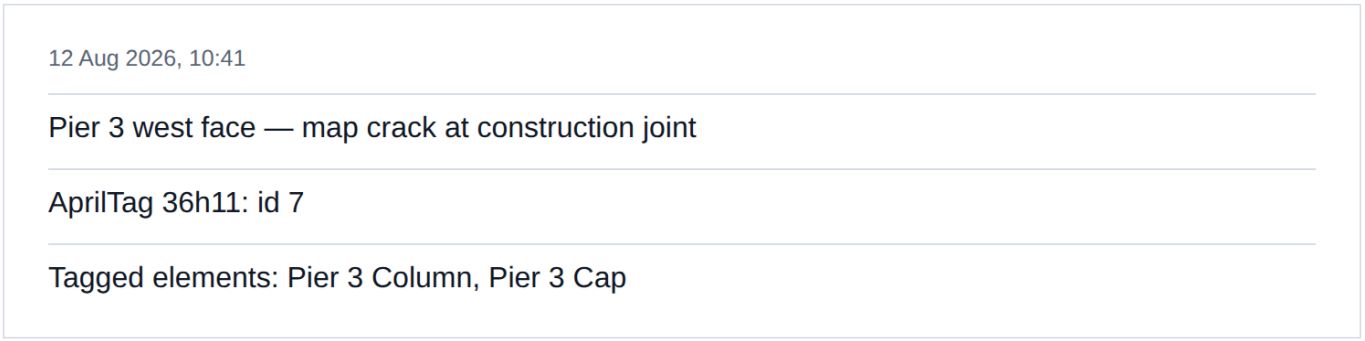


The selection status line.

Set photo location from camera does the opposite of Snap view to me: it writes the 3D camera's current position back onto the photo. Use it when you know where you were standing better than the GPS did — position the 3D view at the real spot and push that position onto the photo.

Where the tags end up

Tagged element names are shown on the photo card and carry through into the exported report, alongside the timestamp, any comment, and any AprilTag detected in the frame.



A photo card's metadata: time, comment, detected tag, and the linked elements.

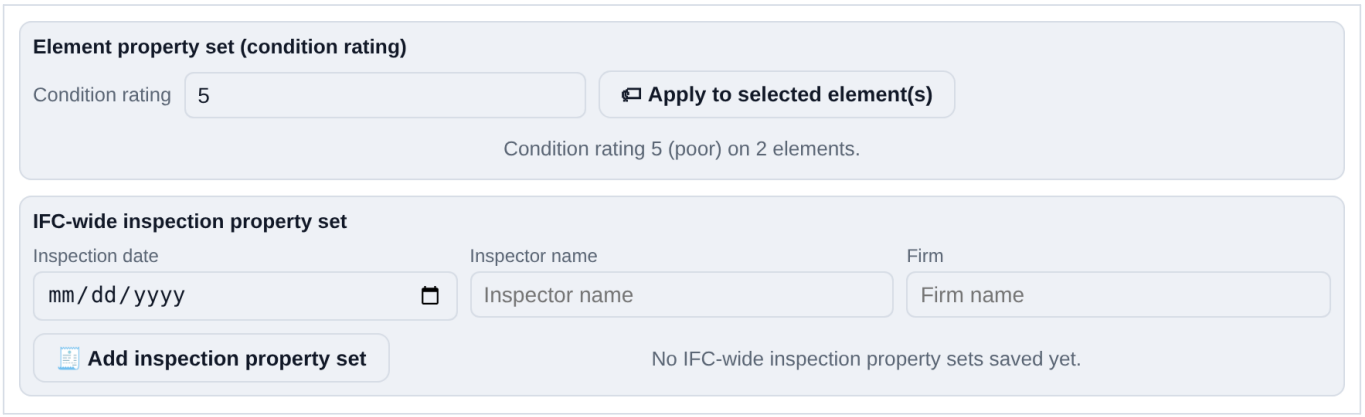
SECTION 9

Condition ratings and compliant IFC export

Tagging a photo to an element says *this is a picture of that column*. Rating an element says *that column is a 5*. The second is what an owner's asset system actually wants, and the app writes it back into the IFC as standard property sets.

Rating elements

1. Open the 3D viewer with the model loaded and click the element, or several.
2. Type the rating in **Condition rating**. It takes an NBI-style number or a word — *6* or *Fair* both work.
3. Tap **Apply to selected element(s)**. The summary line underneath confirms what was written and to how many elements.



Element ratings on top, the model-wide inspection record underneath.

That second card is written once per model rather than per element: inspection date, inspector name and firm. **Add inspection property set** attaches it at project level, so the exported file carries who did the inspection and when.

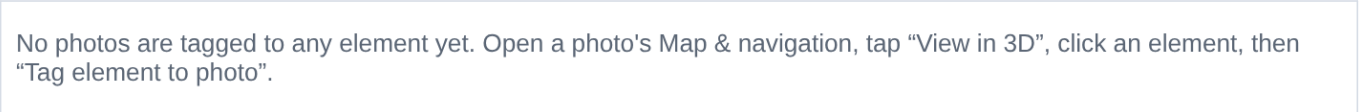
Seeing the ratings

Condition assignment view recolours the model by rating, which turns a column of numbers into something you can read at a glance — and makes it obvious which elements you have not got to yet.

The toggle and its legend: 0-3 critical, 4-5 poor, 6-7 fair, 8-9 good.

Which elements have photographs

Tagged elements on the toolbar lists every element in the bridge that has photos tagged to it, with a thumbnail strip and a count against each. Click a thumbnail to jump to the photo. It is the quickest way to see the coverage you have — and the gaps.



Empty until you tag something; section 8 covers the tagging itself.

Exporting

Export compliant IFC takes the original model text and appends real IFC entities to it — an `IfcPropertySet` plus an `IfcRelDefinesByProperties` per write. Element ratings go out as `Pset_BridgeInspectionElementCondition` attached to the element; the inspection record goes out as `Pset_BridgeInspection` attached to the project.

1. Tap **Export compliant IFC**.
2. Check the preview — it says how many project-level and element-level sets are about to be written.
3. Confirm. The file downloads as *<bridge>-inspection-psets-<date>.ifc*.

> **It needs the original file, and something to write**
>
> The export re-reads the source IFC text so the output is the real model with your property sets added, not a stripped-down re-emit — so have the original to hand. And it refuses to run with nothing to export: add at least one condition rating or the inspection metadata first, or you get *No IFC property sets to export yet*.

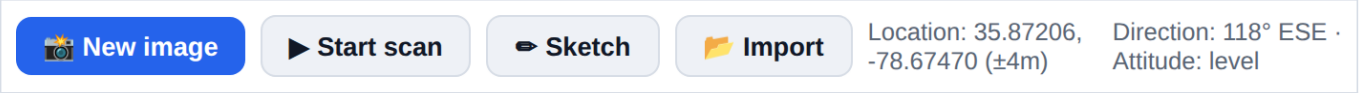
SECTION 10

Geolocating photos

Every photo is stamped with the position and the direction the phone was pointing when it was taken. That is what lets a photo be placed on a map, matched against the model, and found again on the next inspection cycle.

What is captured

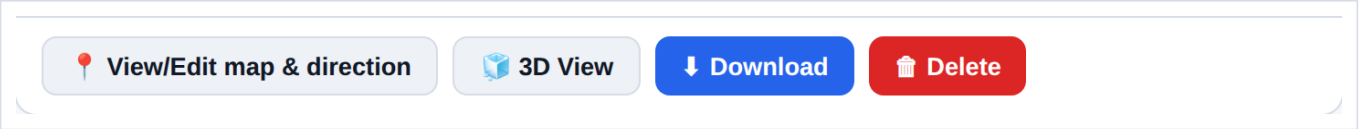
- **Position** — latitude, longitude and the accuracy the phone reported, from the same GPS watch the AR view uses.
- **Direction** — compass heading, as degrees and a cardinal point.
- **Attitude** — whether the phone was level, tilted up or tilted down.
- **Time** — capture timestamp.



The toolbar shows exactly what will be stamped onto the next photo. Check the accuracy figure before a shot you care about.

Reviewing and correcting it

Each photo card carries its own actions. **View/Edit map & direction** puts the photo on a map where you can drag the position and swing the direction arrow — which is how you fix a shot taken on a poor fix, or one taken with the phone in a pocket. **3D View** opens the model from that photo's viewpoint.



Per-photo actions.

> **Correct the fix, not the photo**
>
> If the accuracy figure was poor for a whole run of photos, it is usually faster to fix the position on the map afterwards than to stand around waiting for a better fix at the time. The direction arrow is normally right even when the position is not.

Seeing the whole day at once

Map summary in the header opens every geolocated photo in the bridge on one satellite map. Each photo is an arrow pointing the way the camera was facing, so the coverage pattern is visible at a glance — which faces you worked and which you circled without shooting. Click a marker to open the photo with its metadata beside it.

It also draws the loaded IFC's footprint and its projection onto the map, and restores the bridge's CAD overlay underneath — so photo positions can be read against the model and the drawing rather than against bare

imagery. Overlays are covered in section 17.

Map summary sits next to the build number in the header.

SECTION 11

AprilTags: scale and measurement

An AprilTag is a printed fiducial marker. Put one of known size in the frame and the photo carries its own scale bar — you can measure off the image afterwards without having held a tape against the defect.

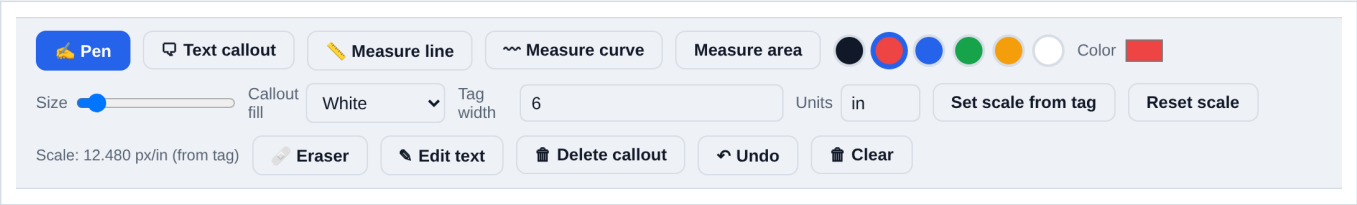
Detection

The app detects the **36h11** family. Detection runs live on the camera preview, so you can see before you press the shutter whether the tag is being picked up, and it runs again on the captured image and on imported images. Detected IDs are stored with the photo.

Live detection status over the camera preview. It reports the tag IDs it can see, or that it sees none.

Setting scale and measuring

1. Print a 36h11 tag and note the width of its **black square** — not the white paper margin around it.
2. Photograph the defect with the tag in frame, as close to the same plane as the surface you want to measure as you can manage.
3. Open the photo and choose annotation.
4. Enter the tag width and its units in the toolbar, then tap **Set scale from tag**. The scale readout will show the resolved pixels-per-unit.
5. Use **Measure line**, **Measure curve** or **Measure area** on the image. Results come out in the units you entered.



The annotation toolbar. Tag width and units on the middle row, the measurement tools on the top row, and the resolved scale reported at the bottom left.

Reset scale clears the calibration. Pen, text callouts, colours and the eraser are all available on the same toolbar, and the overlay is stored separately from the photo — the original image is never modified, and you can

remove the overlay later.

> **Accuracy depends on the plane**
>
> Scale from a tag is exact only in the tag's own plane. A tag lying flat on a deck will not correctly scale a crack on a vertical face, and a tag well in front of or behind the surface of interest introduces perspective error. Get the tag onto, or as near as possible to, the surface you are measuring.

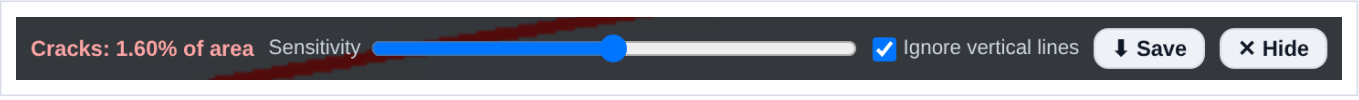
When you cannot place a target

For surfaces where you cannot fix a large physical target, the app also has a **grid rectify** tool: project a laser dot grid onto the surface, drag the on-screen grid nodes onto the dots, enter the real dot spacing in millimetres, and the image is un-skewed into a metric orthophoto. If your laser's spec sheet gives a corner-to-corner fan angle, enter it — a grid laser emits equal *angles*, so its dots spread as tan(angle) on a flat surface rather than evenly, and supplying the fan angle removes that error.

SECTION 12

Automatic crack detection

Every photo card carries a **Cracks** button. It runs a detector over the image, paints what it finds in red on top, and reports the cracked area as a percentage of the frame.



The control strip, over the photo. The percentage is live — it re-runs as you move the sensitivity slider.

- **Sensitivity** — the whole point of the slider is that you sweep it. Wind it up until noise appears, back off until only the real features survive, and read the number there.
- **Ignore vertical lines** — on by default. Form lines, lift joints and conduit runs are vertical and get picked up as cracks otherwise. Turn it off when you are actually looking at vertical cracking.
- **Save** — composites the red overlay onto the full-resolution photo and downloads it. The original photo in the app is untouched.
- **Hide** / **Show** — toggle the overlay so you can compare against the bare image.

It runs on whichever version of the photo is on display, so if you have rectified a shot with the grid tool the detector sees the rectified image — which is the one you want, because in a rectified frame the percentage means something geometrically.

> **A screening aid, not a measurement**
>
> The percentage is cracked *area of the frame*, so it depends on how close you stood and what else is in shot. It is good for ranking which faces need attention and for showing that something changed between cycles at the same standoff. It is not a crack width, and it should not go into a report as one. For widths, put an AprilTag in frame and measure off the photo — section 11.

SECTION 13

Pier scanning for photogrammetry

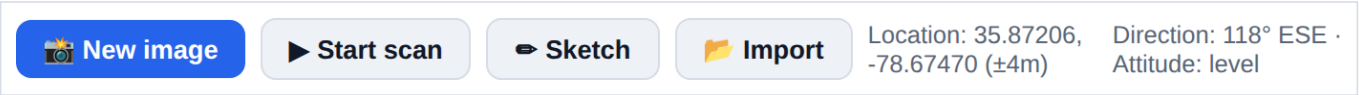
Pier scanning captures an overlapping image set of a surface for reconstruction in a photogrammetry pipeline. Rather than making you judge overlap by eye, the app watches the motion between frames and fires the shutter itself when you have moved the right amount.

How it works

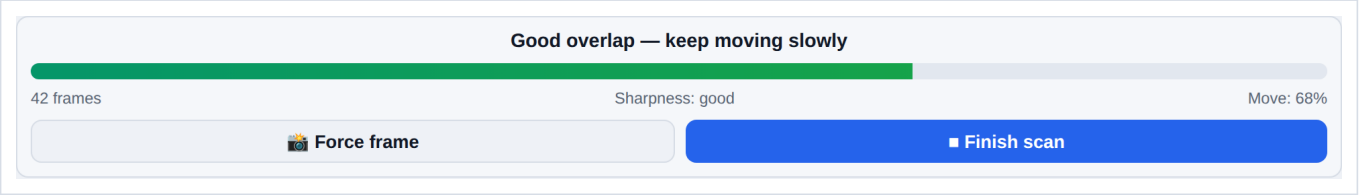
- It tracks frame-to-frame motion at about 11 Hz and takes a shot once the view has moved roughly a quarter of a frame — an overlap target of about 75%.
- Every candidate frame is checked for sharpness, and blurred frames are rejected rather than saved.
- Zoom, focus and exposure are locked at the start of the session where the camera supports it, so the whole set shares one set of intrinsics. This matters a great deal to the reconstruction.

Running a scan

1. Tap **Start scan** on the main toolbar.
2. Let the camera settle and the locks apply.
3. Pan slowly and steadily across the surface, keeping roughly the same standoff distance. Watch the guidance line — it will tell you if you are moving too fast.
4. Use **Force frame** if you want a shot the automatic trigger did not take.
5. Tap **Finish scan** when you have covered the surface.



Start scan on the main toolbar.



The scan HUD: guidance, frame count, a sharpness verdict, and how far you have moved towards the next automatic frame.

Getting the set out

Finished scans are listed under **Pier scans** on the main screen with a thumbnail strip, the frame count, the capture time and the image dimensions, and a note of whether the camera was successfully locked. **Download COLMAP set** exports the frames in the layout COLMAP expects, ready to feed straight into a reconstruction.

Technique matters more than the app

Keep the standoff distance constant, keep the surface filling the frame, move slowly and steadily, and avoid changing your own shadow across the surface as you go. Two slow passes at different heights beat one fast pass. The automatic trigger cannot rescue a set shot too quickly from too many distances.

SECTION 14

Stereo capture and depth maps

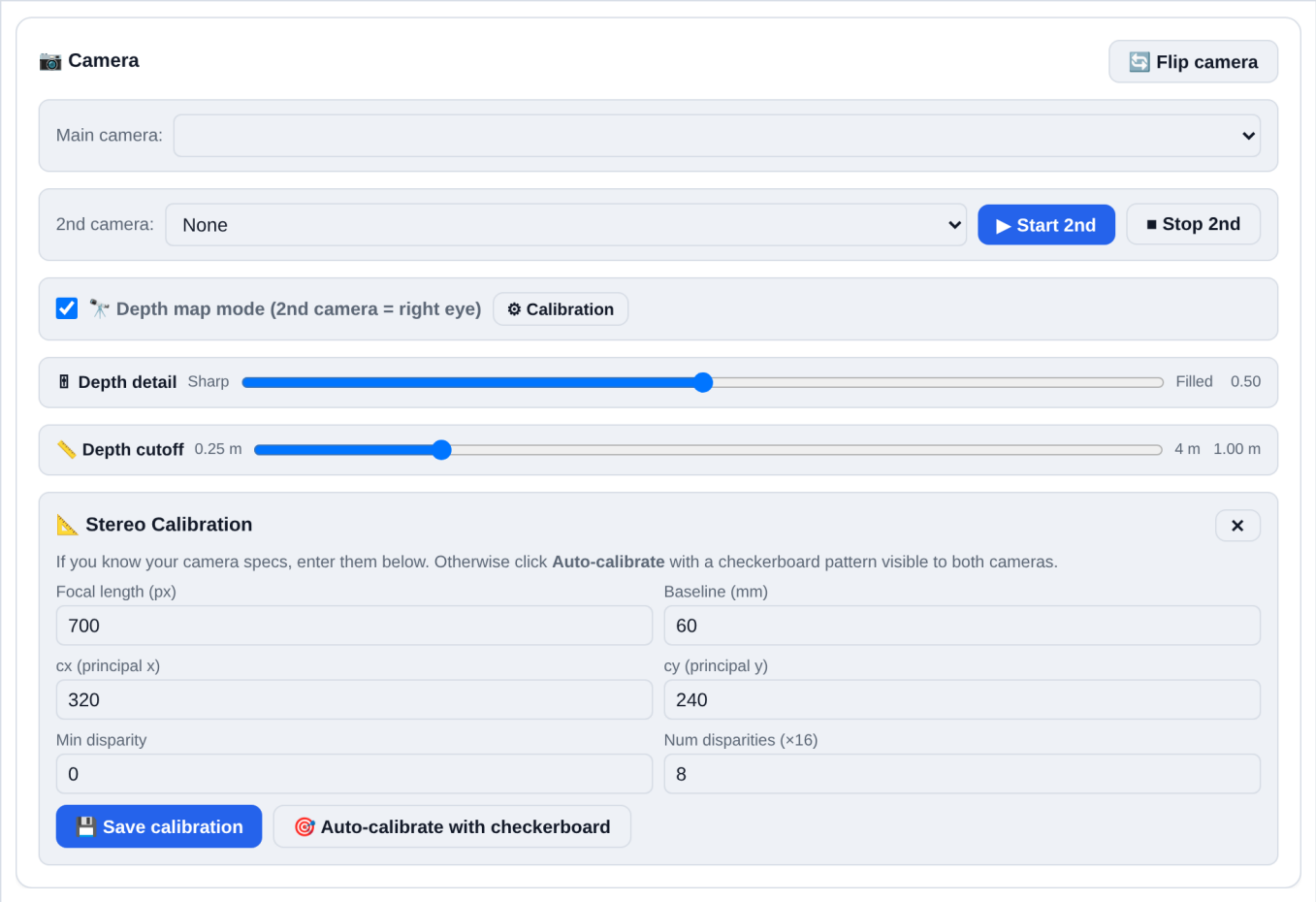
With a second camera attached, the app can capture a stereo pair and turn it into a depth map and a point cloud stored alongside the photograph. This is the most involved feature in the app and the one with a hard external dependency, so read the box below before you plan a day around it.

This needs a depth server running

The stereo pipeline does not run in the browser. It talks to a local depth server over a WebSocket on `localhost:8765`, and that server is a separate program that is **not** shipped in this repository. Without it, Depth map mode connects to nothing and the depth controls do nothing. Everything else in this guide works standalone; this section does not.

Setting the cameras up

1. Open **Settings**, then the **Camera** card.
2. Pick the **Main camera** if the device offers more than one. **Flip camera** swaps front and rear.
3. Pick the **2nd camera** and tap **Start 2nd**. Its preview appears beside the main one.
4. Tick **Depth map mode**. The second camera is treated as the right eye, so mount it to the right of the main one.

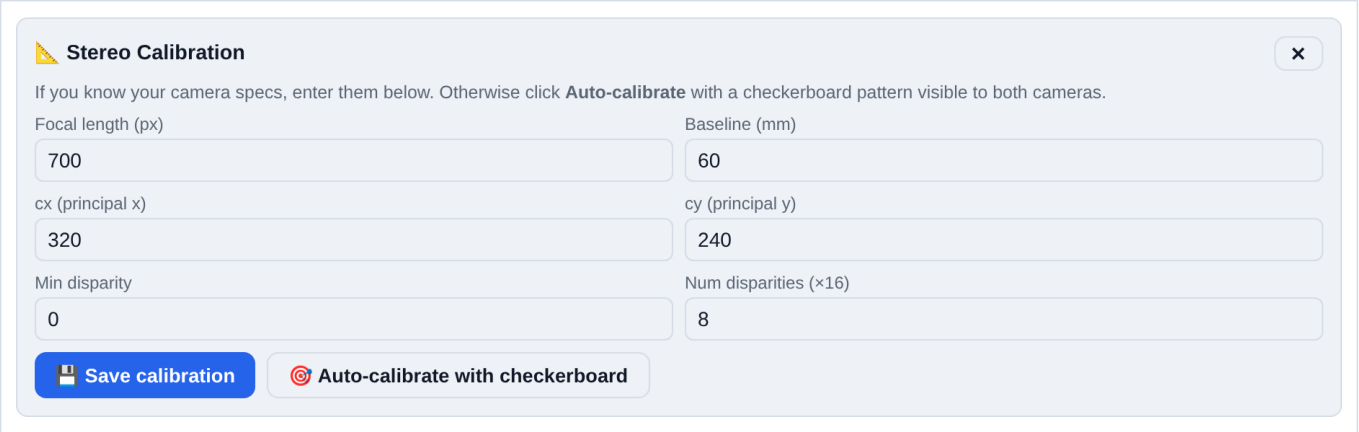


The Camera card with a second camera running and depth mode on.

- **Depth detail** — sharp against filled. Sharp keeps edges honest and leaves holes where the match failed; filled interpolates across them and looks better at the cost of inventing surface.
- **Depth cutoff** — 0.25 m to 4 m. Everything beyond it is discarded, which keeps a pier face from being buried under the background behind it.

Calibrating the pair

Stereo depth is metric only if the geometry of the pair is known. Two ways to supply it:



Enter known intrinsics, or let the checkerboard routine solve for them.

- **Type the numbers** — focal length in pixels, baseline in millimetres, the principal point, and the disparity search range. Use this when you have a spec sheet or a previous calibration.
- **Auto-calibrate with checkerboard** — show a **9×6 inner-corner** checkerboard (that is 10×7 squares) to **both** lenses at once. It collects 15 frames; tilt and move the board between them rather than holding it still, and fill different parts of the frame.

Baseline is the measurement people get wrong: it is the distance between the two lens centres, and an error there scales every depth you produce.

What a capture stores

In depth mode a capture writes three things against one record — the photograph, the depth map as an image, and a PLY point cloud. All three go into the bridge ZIP, the depth map under `images/` and the cloud under `pointclouds/`, so the pair can be taken into whatever you use downstream. The report generator can also export the depth map instead of the photo for a given figure, which is occasionally the clearer illustration of a spall.

SECTION 15

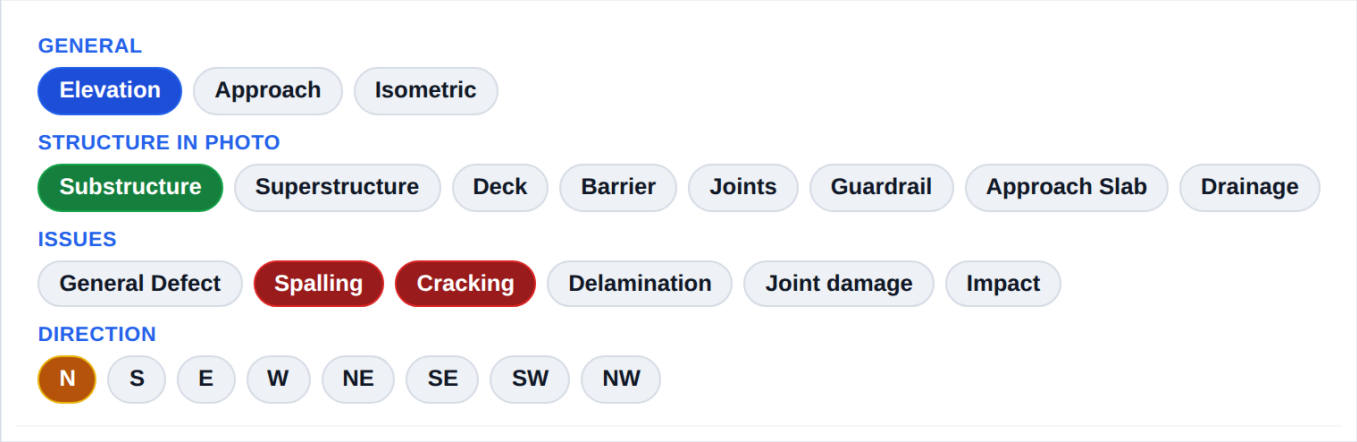
Report logging

Photos belong to a bridge, and the report is generated per bridge, from the photos in it. The generator is not a dumb photo dump: it sorts the photos into report sections and writes a caption for each one, both driven by the tags you put on the photo. Tagging as you shoot is what makes the report come out right.

Tag as you go

Four tag groups. Two of them decide the report's structure.

- **General** - Elevation, Approach, Isometric. Any of these sends the photo to the front of the report.
- **Issues** - General Defect, Spalling, Cracking, Delamination, Joint damage, Impact. Any of these sends the photo to the defects section.
- **Structure in Photo** - Substructure, Superstructure, Deck, Barrier, Joints, Guardrail, Approach Slab, Drainage. Recorded on the photo, not used for sorting.
- **Direction** - the eight compass points. Feeds the “looking North” half of a general-view caption and the arrow on the location map. If you leave it blank the photo’s own compass heading is used, so it is only worth setting when the heading is wrong or missing.



The tag picker. Tags are multi-select - a photo can be both Substructure and Spalling.

How photos are sorted and captioned

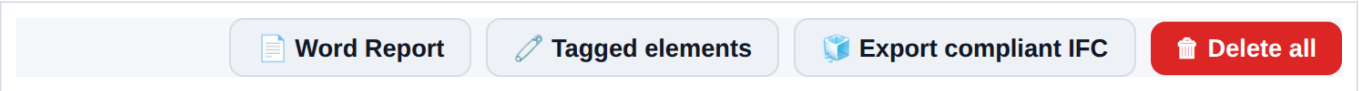
Each photo lands in exactly one section, in this priority order:

- **General Views** - anything with a General tag, ordered Elevation, then Approach, then Isometric, then by capture time. Caption: “Bridge Elevation View looking North” and the like, built from the General tag and the direction.
- **Observed Defects** - anything left with an Issues tag. Caption: **your comment, used verbatim**. With no comment it falls back to “Defect: Spalling, Cracking” - readable, but not a finding. This is the single highest-value habit in the app: write the comment on the defect photo and the report writes itself.

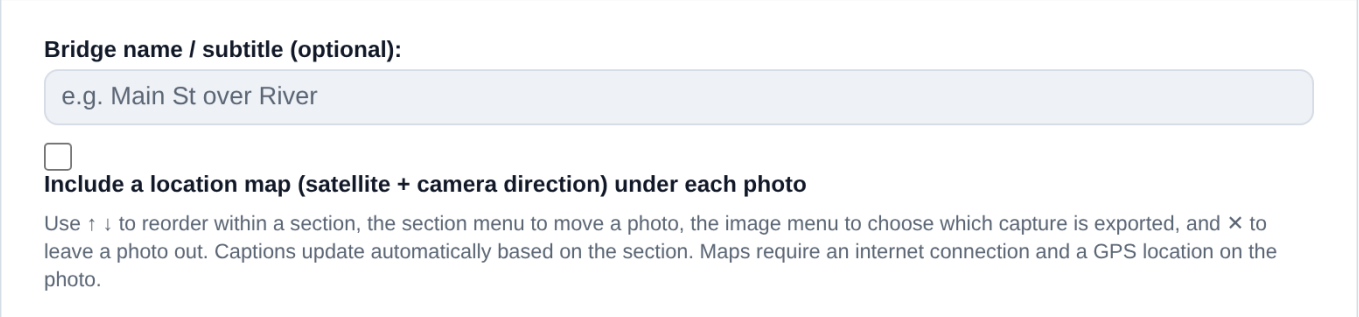
- **Additional Photographs** - everything else. Caption: the comment, else “View looking Southwest”, else “Site photograph”.

Preview and reorder before generating

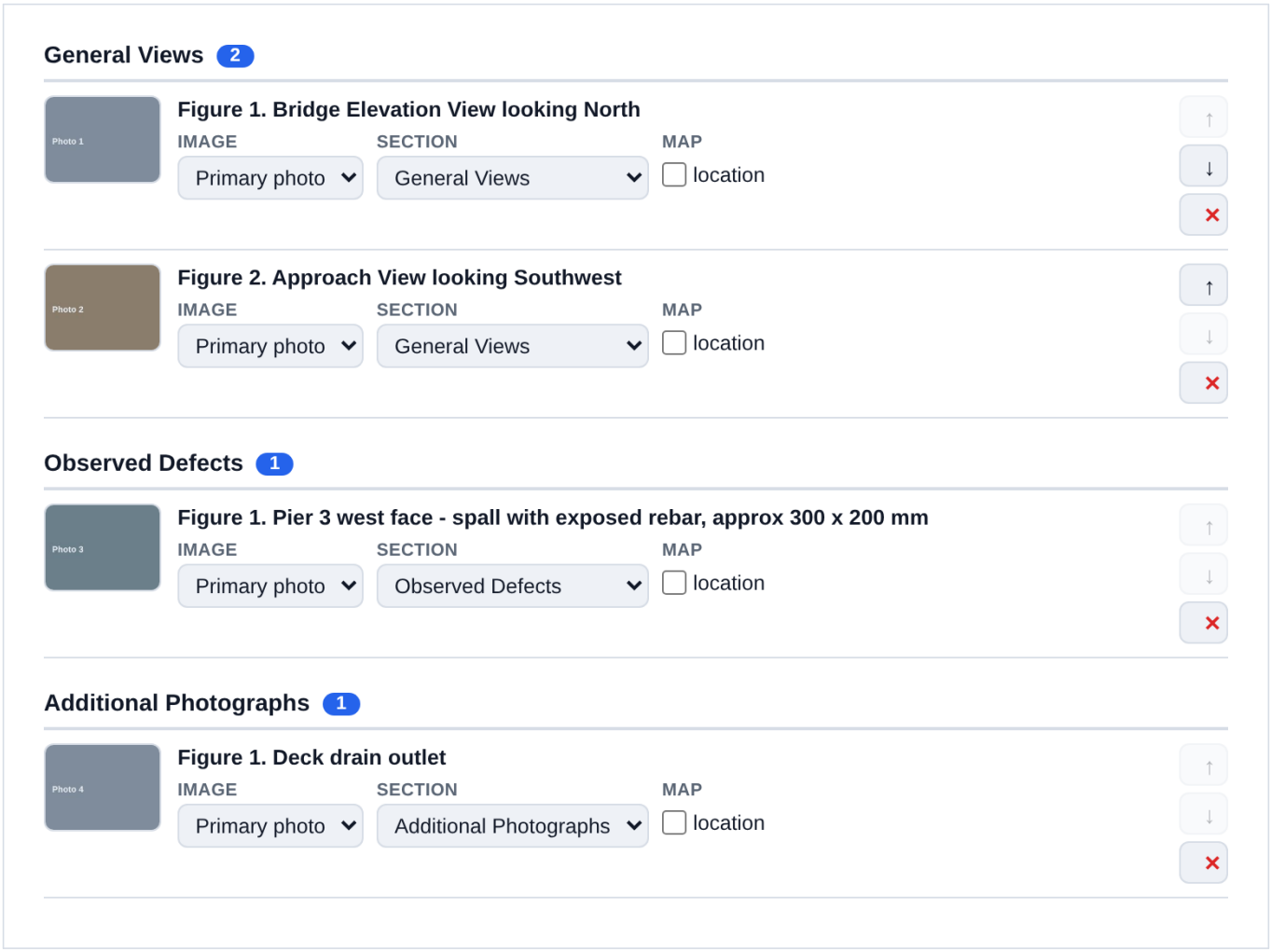
1. Tap **Word Report** on the toolbar. It opens a preview rather than generating straight away.
2. Set the bridge name / subtitle if you want one on the title block.
3. Tick **Include a location map** to put a satellite thumbnail with a camera-direction arrow under every photo.
4. Adjust the running order: the arrows reorder within a section, the section menu moves a photo to a different one, the image menu picks which capture is exported when a photo has more than one, and the cross leaves a photo out entirely.
5. Tap **Generate .docx**.



The report toolbar.



Title-block subtitle and the location-map toggle.



The ordering view. Note the captions: they are derived, and they re-derive when you move a photo to another section - drop a general view into Observed Defects and its caption becomes the comment.

Save the layout

Save layout stores the running order on the bridge. Next time you open the report it is reused, as long as the set of photos is exactly the same. Add or delete a photo and the layout is rebuilt from the tags, so save the layout last.

What comes out

A .docx, generated entirely on the device - nothing is uploaded. US Letter, one-inch margins, Calibri, images fitted to about 480 by 600 points, figures numbered per section. It opens in Word for editing, which is the point: the app produces the photo log, you write the assessment around it.

Location maps need a connection

The map thumbnails are Esri satellite tiles at zoom 18 with the roads and place-names overlays composited on top, fetched at generation time. Generate the report in signal. Without a connection each map degrades to a schematic direction arrow on a plain background rather than failing the export, and a photo with no GPS location gets no map at all.

The raw archive

Download ZIP on the bridge banner is the other half of reporting. It writes out every image - primary, secondary, depth map, deskewed versions, annotation overlays - plus point clouds, any KML overlay, and both a `metadata.json` and a `metadata.csv` carrying comment, tags, location, heading, attitude and AprilTag detections for every photo. That is the file to keep for the record, and the one to hand to anyone who wants the data rather than the document.

SECTION 16

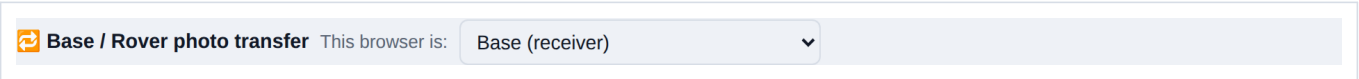
Working as a pair: transferring between devices

The intended two-person setup: the lead inspector works the structure with a phone, taking photographs; a note taker follows with a tablet, receiving each photo as it is taken and writing it up on a bigger screen while the lead keeps moving. The link is direct between the two devices over WebRTC - the photos never touch a server.

Roles

The receiver is the Base

This trips people up. **Base** is the *receiver* and drives the pairing; **Rover** is the *sender*. So the note taker's tablet is the Base, and the lead inspector's phone - the one actually taking pictures - is the Rover.

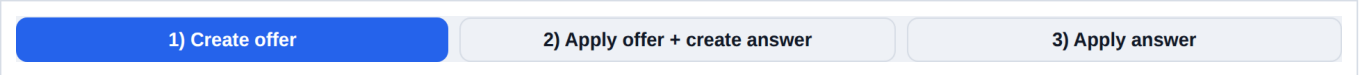


Set the role first. Everything else is greyed or refused until the role matches what you are trying to do.

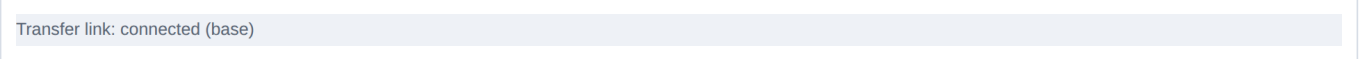
Pairing the two devices

There is no signalling server, so the two devices have to exchange their connection descriptions by hand. The QR route is much faster than typing.

1. Both devices: open **Transfer** and set the role - tablet to Base, phone to Rover.
2. **Tablet (Base):** tap **1) Create offer**, then **Show local as QR**.
3. **Phone (Rover):** tap **Scan QR to remote** and read the tablet's code, then **2) Apply offer + create answer**, then **Show local as QR**.
4. **Tablet (Base):** tap **Scan QR to remote**, read the phone's code, then **3) Apply answer**.
5. Watch the status line settle on **Transfer link: connected**.



The three pairing steps, in order. Each device only uses the ones for its role.



Connected. The log pane underneath records every state change and every file, which is where to look when it does not.

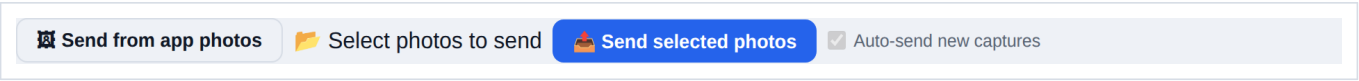
If a camera will not read the code, **Copy local SDP** and paste it into the other device's **Remote SDP** box by any means you like. **Reset transfer session** tears everything down so you can start the sequence again, which is usually quicker than debugging a half-open link.

Sending photos

The moment the link opens, the Base sends the active bridge across and the Rover opens it - creating it locally first if the phone has never seen it. Both devices are then filing into the same bridge. **Sync bridge to rover** re-sends it if the lead switches bridges.

On the phone, the sending device:

- **Auto-send new captures** - the setting that makes the workflow work. Every photo goes across as it is saved, with no extra tap. The choice is remembered between sessions.
- **Send from app photos** - pick from what is already stored, for catching the tablet up on a backlog.
- **Select photos to send** - send files from the device that were never captured in the app.



The sender controls. These appear only in the Rover role.

Files are chunked with backpressure handling, so a big photo will not swamp the channel, and each one is hashed - re-sending the same image to the same bridge is skipped rather than duplicated.

> **Only the image crosses the link**
>
> This is the one thing to plan around. The transfer carries the image bytes and the capture time - **not** the GPS location, the heading, the tags, or the AprilTag detections. A received photo lands on the tablet with no position, no tags, and a placeholder comment of "Transferred from rover: …". So a report generated on the tablet will have no location maps and no "looking North" captions for transferred photos unless the note taker fills them in.

So decide which device holds the record

Two workable patterns, and it is worth picking one before you start rather than discovering the difference at the end of the day:

- **Tablet writes the report.** The note taker is going to be typing anyway - so they type the comment, which becomes the defect caption verbatim, and set the tags, which decide the sections. Accept that transferred photos carry no GPS. This is the faster pattern and it is what the live link is for.

- **Phone holds the record.** Use the tablet as a live second screen for review only, and at the end of the day move the whole bridge across with **Download ZIP** on the phone and **Import ZIP** on the tablet. That path is lossless - locations, headings, tags, comments, AprilTags, annotations and overlays all come with it - and it is the one to use when the report needs location maps.

> **It works with no signal**
>
> The link is peer-to-peer. It tries a public STUN server while gathering candidates, but gives up after eight seconds and proceeds regardless - so two devices on the same Wi-Fi, or on one phone’s hotspot, pair and transfer with no internet at all. Which is the point, under a bridge.

Part three

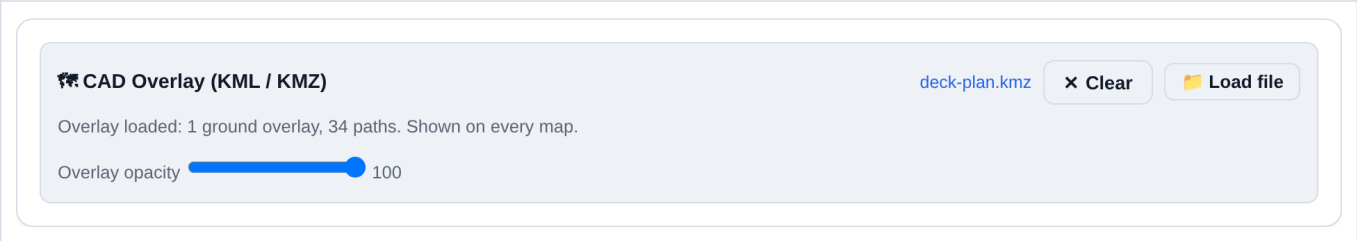
The rest of the toolbox

Smaller things that do not belong to any one workflow, but that you will want at some point: getting your own drawings onto the maps, drawing a sketch when a photograph will not do, installing the app properly, and getting an error log out of it when something misbehaves.

SECTION 17

CAD overlays (KML / KMZ)

You can drop your own drawing onto every map in the app — a general arrangement, a deck plan, a scour survey, anything you can get out of CAD or GIS as KML or KMZ. Once loaded it appears under the bridge map, under the photo location editor, and on the map summary, so photo positions can be read against the drawing rather than against bare satellite imagery.



The CAD Overlay card in Settings.

1. Open **Settings** and find **CAD Overlay (KML / KMZ)**.
2. Tap **Load file** and pick a .kml or .kmz.
3. Set the opacity so the imagery still reads underneath. The setting is remembered.

What it handles

- **KMZ** is unpacked in the browser. Images inside it are extracted and embedded, so a KMZ with a raster ground overlay works offline afterwards — nothing is fetched at draw time.
- **Ground overlays** both ways: an axis-aligned `LatLonBox`, including its rotation, and a four-corner `gx:LatLonQuad` for a drawing that is not north-up.
- **Vector features** — paths and placemarks drawn over the map.

The overlay is stored against the bridge, so each structure carries its own drawing and switching bridges switches drawings. It is included in **Download ZIP** — the original file where one was supplied, so what comes out is what went in.

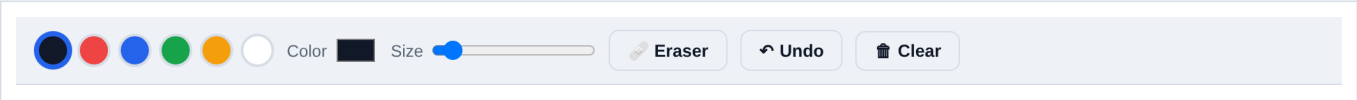
> **Georeference it before you load it**
>
> The app places the overlay exactly where the KML says to. If the drawing lands in the wrong field, the problem is upstream in whatever exported it, and no control here will fix it. Check it against the satellite basemap the first time you load it, at low opacity, before you rely on it in the field.

SECTION 18

Sketches, installing, and the debug console

Sketches

Sketch on the main toolbar opens a blank canvas. Draw with a finger or a stylus — swatches, a custom colour, brush size, eraser, undo, clear.

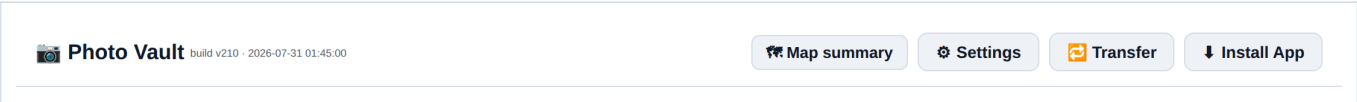


The sketch toolbar.

The important part is what happens on save: a sketch is stored as a **photo record**, with the same comment, tags and location as a photograph. So it sorts into the report through exactly the same rules — tag it with an Issue and it lands in Observed Defects with your comment as its caption. Use it for the things a camera cannot capture: a crack map across a whole face, a sketch of what is behind the fascia, a detail you can see but not photograph.

Installing the app

Install App appears in the header when the browser offers it. Installing makes it a standalone app with its own icon, and it is what makes offline reliable rather than merely likely — an installed PWA keeps its cache and its stored data rather than competing with the browser's own housekeeping.



The header: build number beside the title, the map summary link, Settings, Transfer, and Install App when it is available.

The build number next to the title is worth knowing where to find. It is the first thing to check when something described in this guide is not where it should be — see section 2.

The debug console

Errors that would normally land in a browser console are invisible on a phone. **Show error log** in Settings puts them on screen in a panel at the bottom, with **Copy** to put the whole log on the clipboard and **Clear** to start a fresh one.

Off by default. Turn it on before reproducing a problem, not after.

When something misbehaves in the field, the useful sequence is: turn the log on, do the thing that fails, copy the log, and send it with a note of the build number. That is usually enough to identify the fault without anyone having to reproduce it on the structure.